



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

ANISH NAIR

BATCH: A1

PRN:25070122037

Subject: Programming with Java

ASSIGNMENT No: 6

TITLE :Write a Java program to demonstrate the use of an inner class, an anonymous class.

Problem Statement

Write a Java program to demonstrate the use of an inner class, an anonymous class.

Learning Objectives

To implement inner classes and anonymous classes for improving encapsulation and event handling in Java.

Theory

An inner class is a class defined inside another class and has access to all members of the enclosing class, including private members. Inner classes help improve code organization and logical grouping of related classes. Anonymous classes are unnamed inner classes that are instantiated only once and are mainly used for implementing interfaces or abstract classes. They simplify code by eliminating the need for separate class definitions. These concepts are widely used in Java GUI programming and event handling

Outcome

```

6    class FoodOrder {
20    }
21
22
23    public class FoodDeliveryDemo {
24        public static void main(String[] args) {
25            FoodOrder order = new FoodOrder("Anish", "Chicken Biryani");
26
27            FoodOrder.OrderDetails details = order.new OrderDetails();
28            details.display();
29
30            StatusUpdate preparing = new StatusUpdate() {
31                @Override
32                public void updateStatus() {
33                    System.out.println("Status: Order is being prepared.");
34                }
35            };
36
37            StatusUpdate outForDelivery = new StatusUpdate() {
38                @Override
39                public void updateStatus() {
40                    System.out.println("Status: Order is out for delivery.");
41                }
42            };
43
44            StatusUpdate delivered = new StatusUpdate() {
45                @Override
46                public void updateStatus() {
47                    System.out.println("Status: Order has been delivered.");
48                }
49            };
50
51            preparing.updateStatus();
52            outForDelivery.updateStatus();
53            delivered.updateStatus();
54        }
55    }

```

```

1  // ===== File: FoodDeliveryDemo.java =====
2  interface StatusUpdate {
3      void updateStatus();
4  }
5
6  class FoodOrder {
7      String customerName;
8      String item;
9
10     FoodOrder(String customerName, String item) {
11         this.customerName = customerName;
12         this.item = item;
13     }
14
15     class OrderDetails {
16         void display() {
17             System.out.println("Customer: " + customerName);
18             System.out.println("Item Ordered: " + item);
19         }
20     }
21 }
22
23 public class FoodDeliveryDemo {
24     public static void main(String[] args) {
25         FoodOrder order = new FoodOrder("Anish", "Chicken Biryani");
26
27         FoodOrder.OrderDetails details = order.new OrderDetails();
28         details.display();
29
30         StatusUpdate preparing = new StatusUpdate() {
31             @Override
32             public void updateStatus() {
33                 System.out.println("Status: Order is being prepared.");
34             }
35         };
36
37         StatusUpdate outForDelivery = new StatusUpdate() {
38             @Override
39             public void updateStatus() {
40                 System.out.println("Status: Order is out for delivery.");
41             }
42         };

```

Vehicle Name: Royal Enfield

Vehicle Model: Classic 350

Starting the engine... Vroom Vroom!

```

1 interface Action {
2     void perform();
3 }
4
5 class Vehicle {
6     String name;
7     String model;
8
9     Vehicle(String name, String model) {
10         this.name = name;
11         this.model = model;
12     }
13
14     class VehicleDetails {
15         void display() {
16             System.out.println("Vehicle Name: " + name);
17             System.out.println("Vehicle Model: " + model);
18         }
19     }
20 }
21
22 public class VehicleDemo {
23     public static void main(String[] args) {
24         Vehicle vehicle = new Vehicle("Royal Enfield", "Classic 350");
25
26         Vehicle.VehicleDetails details = vehicle.new VehicleDetails();
27         details.display();
28
29         Action startEngine = new Action() {
30             @Override
31             public void perform() {
32                 System.out.println("Starting the engine... Vroom Vroom!");
33             }
34         };
35
36         startEngine.perform();
37     }
38 }

```

Customer: Anish

Item Ordered: Chicken Biryani

Status: Order is being prepared.

Status: Order is out for delivery.

Status: Order has been delivered.

Conclusion : The inner class (VehicleDetails/OrderDetails) accesses the enclosing class's data directly for display, while anonymous classes provide quick, one-time implementations of the Action/StatusUpdate interface without writing separate named classes.

Exercise

1. Create a Vehicle program where an inner class displays vehicle details and anonymous class performs an action.
2. Develop a Food Delivery Application where an inner class handles order details and anonymous classes handle delivery status updates.